

רכיבים קשירים היטב - פתרון

1. כיצד יכול להשתנות מספר הרכיבים הקשירים היטב בגרף אם מוסיפים קשת חדשה?

תשובה:

- הוספת קשת חדשה בגרף יכולה להקטין את מספרי הרכיבים הקשירים היטב שלו בכל מספר k , $1 \leq k \leq n - 1$ (מספר הקודקודים בגרף).

הסבר - אם יש $k + 1$ קודקודים שכל אחד מהם מהווה רכיב קשירות בפני עצמו (או שכל אחד מהם שייך לרכיב קשיר היטב שונה), ויש מסלול פשוט (מכוון) שעובר דרך כל הקודקודים האלה, אזי הוספת קשת אחת שתסגור מעגל תקטין את מספר הרכיבים הקשירים היטב ב- k .

מ- $k + 1$ רכיבים קשירים היטב, יורדים לרכיב קשיר היטב אחד (!)

שהרי ראינו בתרגול שכל הקודקודים שעל מעגל שייכים לרכיב קשיר היטב אחד.

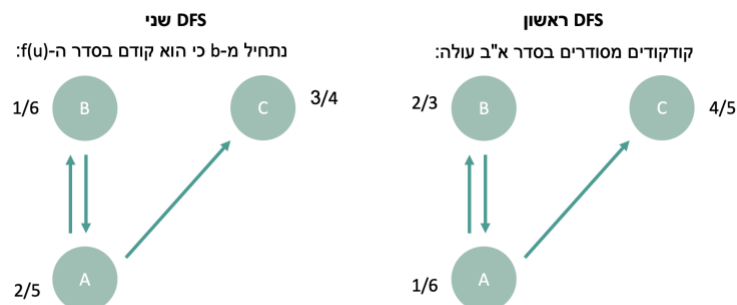
- כמובן שהוספת קשת יכולה גם שלא לשנות את מספר הרכיבים הקשירים היטב בגרף (כגון אם הקשת שנוספה היא בין שני קודקודים שכבר היו שייכים לאותו רכיב קשיר היטב).

- בכל מקרה, הוספת קשת לא יכולה להגדיל את מספר הרכיבים (כי כל זוג קודקודים שהיו שייכים לאותו רכיב קשיר היטב לפני הוספת הקשת החדשה, יהיו עדיין שייכים לאותו רכיב קשיר היטב גם אחרי הוספתה).

2. פרופסור דיוור טוען שניתן לפשט את האלגוריתם למציאת רכיבים קשירים היטב על ידי שימוש בגרף המקורי (ולא בגרף המשוחלף G') בחיפוש לעומק השני, וסריקת הקודקודים בסדר עולה של זמני סיום (כלומר, $f[u]$). האם הפרופסור צודק?

תשובה:

הפרופסור טועה. דוגמא נגדית:



נקבל שכל הקודקודים שייכים לאותו רכיב קשיר היטב, מה שכמובן לא נכון (c לא שייך).

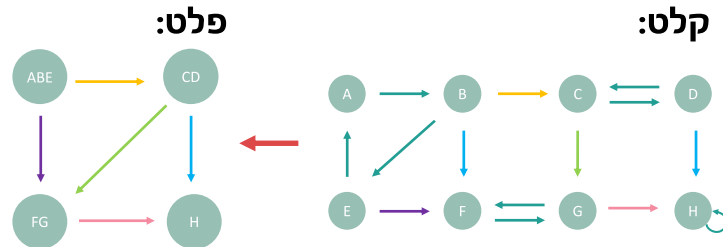
3. כתבו אלגוריתם שזמן ריצתו $O(|V| + |E|)$ לחישוב גרף הרכיבים של גרף מכוון

$$G=(V,E)$$

הבטיחו שתהיה **לכל היותר קשת אחת** בין שני קדקודים בגרף הרכיבים שיוצר האלגוריתם שלכם.

הוכיחו נכונות האלגוריתם (שהוא אכן מבצע את הדרוש).

דוגמה גרפית:



תזכורת:

את גרף הרכיבים של גרף $G=(V,E)$ אנו מסמנים $G^{scc}=(V^{scc},E^{scc})$ כאשר V^{scc} מכילה קדקוד אחד עבור כל רכיב קשיר היטב ב- G ו- E^{scc} מכילה את הקשת (u,v) אם מקדקוד ברכיב הקשיר היטב של G המתאים ל- u , קיימת קשת מכוונת לקדקוד ברכיב הקשיר היטב של G המתאים ל- v

מוטיבציה לתשובה:

- עבור הקדקודים - נרצה לצמצם את כל הקדקודים מאותו רכיב קשירות לקדקוד אחד, לצורך כך נרצה לדעת איזה קודקודים שייכים לאותו רכיב הקשירות ומהם נוכל לבחור אחד ולהוסיף לרשימה V^{scc} .
- עבור הקשתות – נרצה למצוא את הקשתות (u,v) אשר u נמצא ברכיב i ו v ברכיב j , מתוכן נרצה לבחור **קשת אחת** ולהכניס לרשימה E^{scc} .
- סיבוכיות זמן – נרצה לדעת מהם הרכיבים הקשירים היטב לטובת בניית גרף הרכיבים, אז זמן האלגוריתם יהיה לפחות זמן מציאת רכיבים אלו $\Omega(|V|+|E|)$

תשובה: אלגוריתם לחישוב גרף הרכיבים של גרף מכוון $G = (V, E)$:

find SCC_graph(G):

```

1. Gsccl ← new graph()
2. Gsccl_forest ← SCC(G) // get forest graph  $\Theta(V + E)$ 
3. Sorted_forest ← countingSort(G_forest, d) // sort by d field in each Vertex  $\Theta(V)$ 
4. scc ← -1
5. for each  $u \in \text{Sorted\_forest}$  // pick one vertex for each SCC  $\Theta(V)$ 
6. do if  $\text{parent}[u] = \text{NIL}$  // u is one of the SCC roots as no vertex visits u as part of SCC
7. then scc ← scc + 1
8. Gsccl[scc] ← new vertex()
9.  $\text{parent}[u] \leftarrow \text{Gsccl}[scc]$ 
10. scc_ReverseEdges ← new array[MAX] // new array in length of MAX  $\sim |V|$ 
11. for each  $(u, v) \in G$  //  $\Theta(V + E)$ 
12. do if  $\text{parent}[u] \neq \text{parent}[v]$ 
13. then addAdj(scc_ReverseEdges  $\text{parent}[v]$ ,  $\text{parent}[u]$ ) // add the scc of u -  $\text{parent}[u]$  as first element in the
ReverseEdges list of the scc of v -  $\text{parent}[v]$ 
14. scc_Edges ← new array[MA // new array in length of MAX  $\sim |V|$ 
15. for each vertex  $i \in G$  // insert the edges of the scc graph (including edges duplications)
16. do for each vertex  $j \in \text{scc\_ReverseEdges}[i]$ 
17. do addAdj(scc_Edges  $[j]$ ,  $i$ )
18. for each vertex  $i \in \text{Gsccl}$  // remove duplications from the edges lists of the scc graph
19. do for each vertex  $j \in \text{scc\_Edges}[i]$ 
20. if  $j = \text{next}(\text{scc\_Edges}[i], j)$  // j is equal to the next element in the edges list of i
21. then  $\text{scc\_Edges}[i] \leftarrow \text{Remove}(\text{scc\_Edges}[i], j)$  //remove the duplication of j
22. Output(Gsccl)

```

הוכחת נכונות

נרצה להראות שהאלגוריתם יחזיר קדקוד עבור כל רכיב קשיר בגרף G , ולכל היותר קשת אחת (i,j) בין רכיב i לרכיב j

טענה 1:

לכל רכיב קשיר היטב שמתקבל מיער העומק בצעד 2 יהיה ייצוג יחיד בגרף G_{CC} .

הוכחה:

האלגוריתם יעבור על כל קדקודי גרף ובכל פעם שיזהה קדקוד שהוא שורש ($\pi[u] = NIL$) של רכיב קשיר היטב יוסיף קדקוד לרשימת הרכיבים הקשירים היטב בגרף G_{SCC} . מנכונות אלגוריתם SCC ומהגדרת מבנה הנתונים של עץ - נוכל להניח שלכל עץ ביער יש שורש יחיד ושמשפר הרכיבים הקשירים היטב יהיה זהה למספר השורשים ביער.

טענה 2:

בגרף G_{SCC} יהיו קשתות בין רכיבי קשירות שונים ולכל היותר קשת אחת בין שני רכיבים U, V .

הוכחה:

האלגוריתם יסווג כל קדקוד לרכיב אליו הוא שייך ע"י סימון ערך $[]$ באינדקס הרכיב בגרף G_{SCC} אליו הוא שייך. מתכונות DFS שמבקר בכל הקודקודים הנגישים ברצף ובזכות המיון נוכל לוודא שכל קדקודי אותו רכיב יהיו ברצף וכאשר נבקר בקודקוד שערך $[] = NIL$ נדע שעברנו לרכיב חדש.

בלולאה בשורה 11 האלגוריתם מוצא את כל הקשתות שצריכות לעבור בין הרכיבים. לשם כך האלגוריתם עובר על כל קשתות הגרף, וכל קשת ששני הקצוות שלה שייכים לרכיבים קשירים היטב שונים נכנסת למערך העזר באופן שבכניסה j נשמרת רשימת הקשתות שנכנסות לקודקוד j .

לאחר מכן, בלולאה שבשורה 15 ממלאים את מערך המצביעים scc_Edges שמייצג את קשתות הגרף בסדר הרגיל. מאחר שעוברים על מערך העזר לפי הסדר, אנו מטפלים בכל הקשתות שנכנסות ל- j לפני שעוברים לרשימה הבאה [= לרשימת הקשתות שנכנסות לקודקוד הבא]. לכן, אם יש כמה קשתות מ- i ל- j הן כולן יירשמו במערך scc_Edges ברשימה של i אחת אחרי

השנייה, ברצף.

כעת, כאשר בלולאה בשורה 18 האלגוריתם סורק את הרשימות שיש ב- scc_Edges , אם יש ברשימה כמה איברים זהים, הם יופיעו **ברצף** בזה אחר זה, ולכן נוכל לזהות אותם ולמחוק את המיותרים בסריקה ליניארית של כל רשימה.

לסיכום – נקבל את גרף הרכיבים G^{SCC} כאשר בין כל שני רכיבים עוברת לכל היותר קשת אחת.

זמן ריצה: $\Theta(V + E)$

סיבוכיות מקום: $\Theta(V + E)$